

Break it down,  
Build it Better!

# MICROSERVICES

Small Services. Big Impact.

- ✓ Think Small
- ✓ Deploy Fast
- ✓ Scale Smart

## WHAT ARE MICROSERVICES?

Microservices are an architectural style where an application is built as a collection of small, independent services that communicate over a network (usually through APIs).

➔ Instead of one big application (monolith), we split functionality into separate services, each responsible for a specific business capability.

## EXAMPLE: E-COMMERCE APP

### In a monolith:

- User management
- Product catalog
- Shopping cart
- Payment processing
- Order management

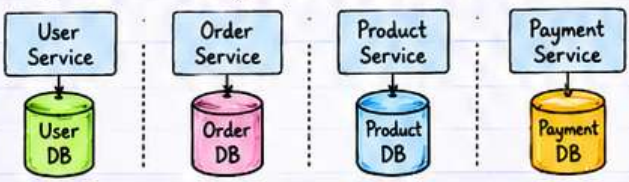
### With microservices:



Each is a separate service.

## KEY CHARACTERISTICS

- 1 INDEPENDENT DEPLOYMENT**  
Each service can be deployed without redeploying the entire system.
- 2 SINGLE RESPONSIBILITY**  
Each service focuses on one business function.
- 3 OWN DATA STORAGE**  
A service manages its own database.
- 4 TECHNOLOGY FLEXIBILITY**  
Different services can use different technologies.



## ADVANTAGES

- ✓ Easier to scale individual components
- ✓ Faster development by multiple teams
- ✓ Better fault isolation
- ✓ Independent deployments
- ✓ Technology flexibility

## CHALLENGES

- ✗ Increased complexity
- ✗ Network latency between services
- ✗ More difficult debugging and monitoring
- ✗ Data consistency can be harder to maintain
- ✗ Requires robust DevOps, logging, and service discovery

## HOW SERVICES COMMUNICATE



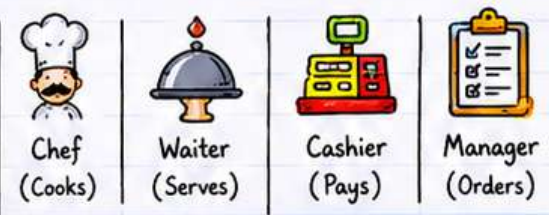
## WHEN TO USE?

- ★ Great for large, complex applications that need to scale and evolve rapidly.
- ★ For small apps, start with a monolith.
- ★ Move to microservices only when necessary.

## ANALOGY: RESTAURANT



Different roles.  
Same goal:  
Happy customers!



## REAL-WORLD USERS

**N** **NETFLIX**  
(Streaming)

**amazon**  
(E-commerce)

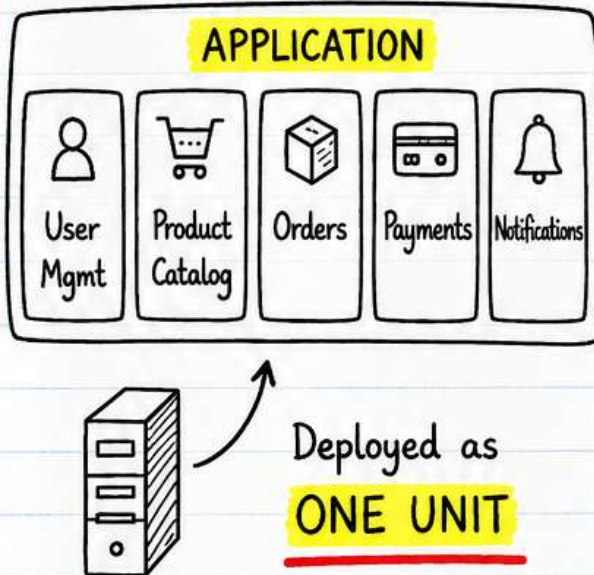
**Uber**  
(Ride-sharing)

**Spotify**  
(Music)

# MONOLITHIC vs MICROSERVICES

## MONOLITHIC ARCHITECTURE

All functionality is built & deployed as a **single unit**.



### Advantages 😊

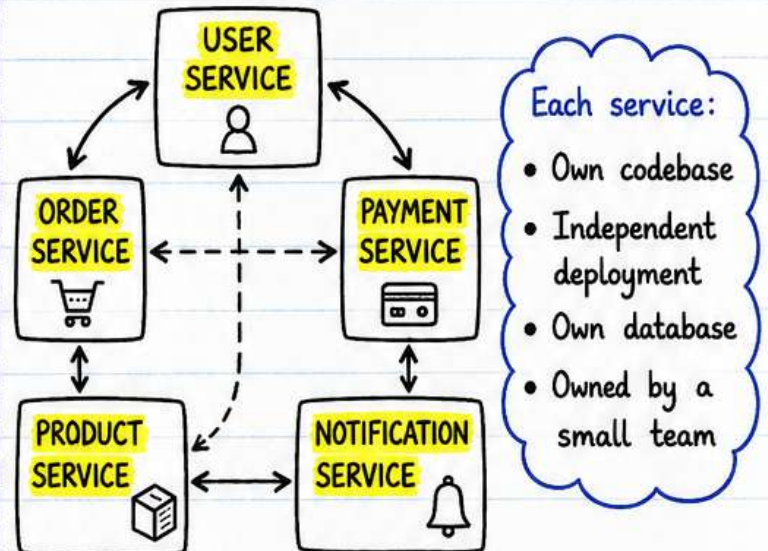
- **Faster** initial development
- **Easier** debugging
- **Simple** deployment
- **Lower** infrastructure cost
- **Easier** local development

### Disadvantages 😞

- Hard to **scale** specific parts
- Large codebase = hard to **maintain**
- **Slower** deployments
- Tech choices affect the **whole system**

## MICROSERVICES ARCHITECTURE

Application is broken into **small, independent** services that communicate over a network.



### Advantages 😊

- Independent **scaling**
- **Faster** deployments
- Better **fault isolation**
- Team **autonomy**
- Tech **flexibility**

### Disadvantages 😞

- Higher operational **complexity**
- Network **latency**
- **Distributed** system challenges
- More **monitoring** needed
- Data **consistency** is harder

## ★ REAL-WORLD EXAMPLES

- Blogs, internal tools, small e-commerce sites often start as **MONOLITHS**.
- Companies like **Netflix**, **Amazon**, **Uber** use extensive **MICRO-SERVICES** to handle massive scale.



### THINK:

Right tool for the right stage of your product!

## WHICH SHOULD YOU CHOOSE?

### 1 START WITH A MONOLITH

if you're building a **new product** with a **small team**.

### 2 MOVE TO MICROSERVICES

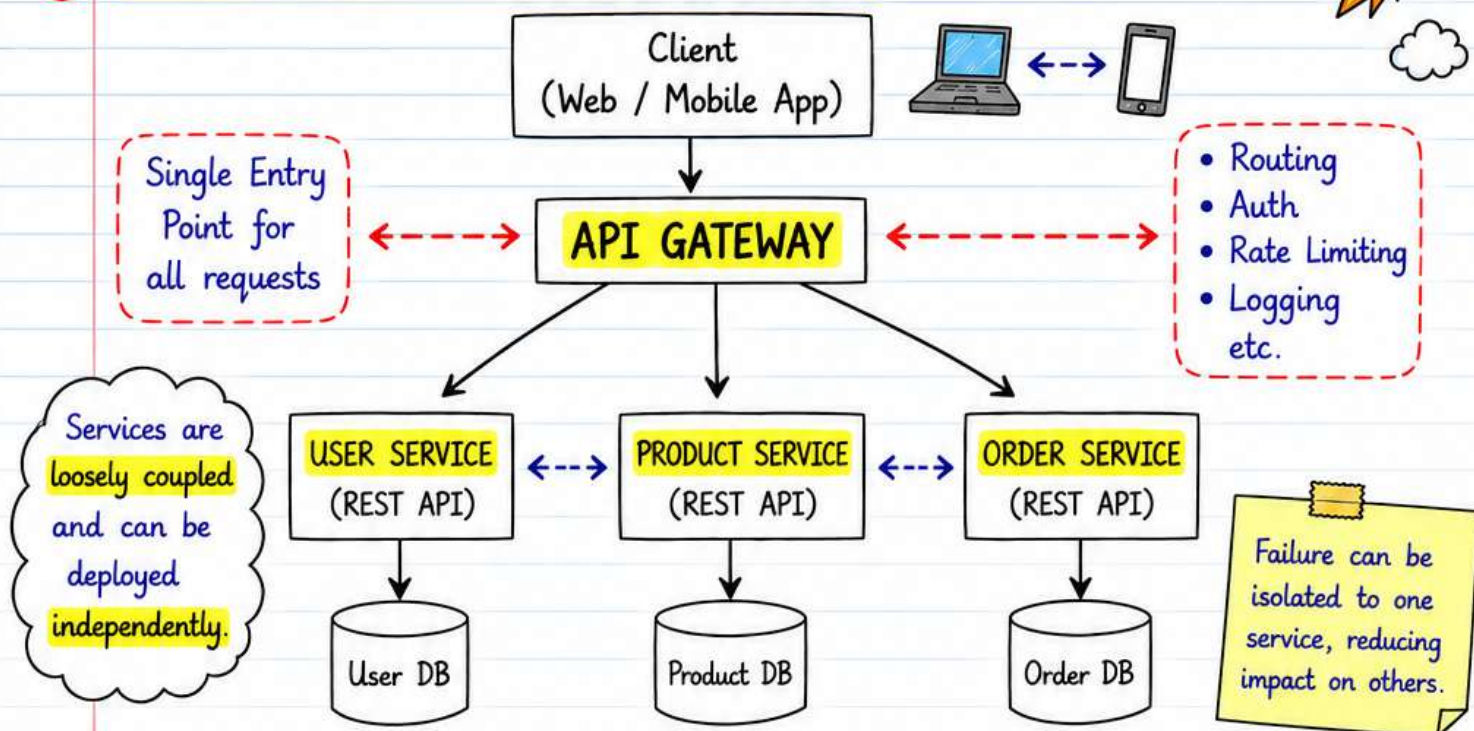
when you hit **scaling**, **deployment** or **team ownership** bottlenecks.



# MICROSERVICES & API GATEWAY

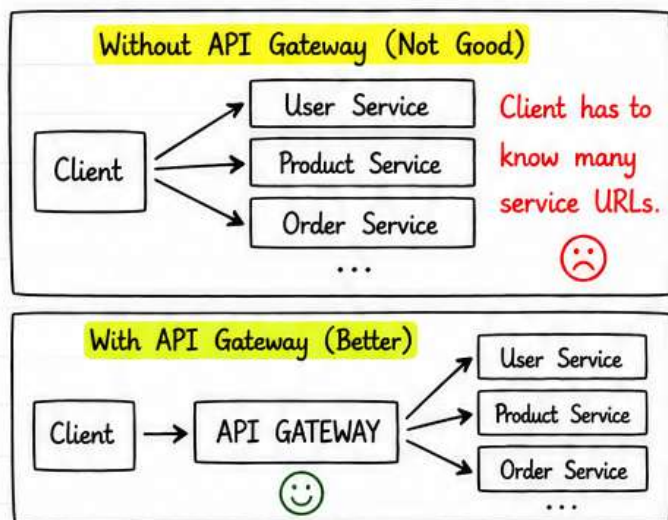


## 2 MICROSERVICES BASIC FLOW DIAGRAM



## 3 WHAT IS AN API GATEWAY?

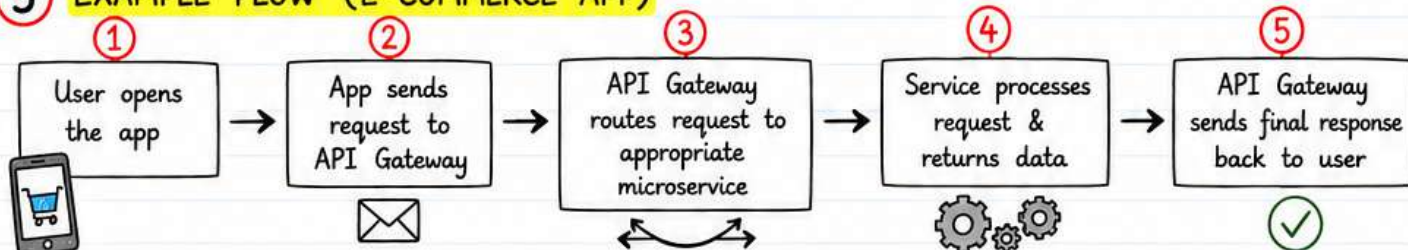
An API Gateway is the **single entry point** for all client requests in a microservices architecture. It acts as a **middleman** between clients and backend services.



## 4 RESPONSIBILITIES OF API GATEWAY

- 1 REQUEST ROUTING → Sends request to the correct microservice.
- 2 AUTHENTICATION & AUTHORIZATION → Verifies users and tokens.
- 3 LOAD BALANCING → Distributes traffic among multiple instances.
- 4 RATE LIMITING → Prevents excessive or abusive requests.
- 5 LOGGING & MONITORING → Tracks API usage, errors and performance.
- 6 RESPONSE AGGREGATION → Combines data from multiple services into one response.

## 5 EXAMPLE FLOW (E-COMMERCE APP)





# CIRCUIT BREAKER

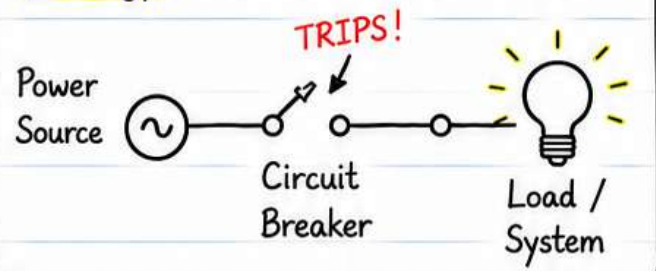


(Design Pattern for Resilient Systems)

## ★ What is it?

**Circuit Breaker** is a design pattern that prevents **cascading failures** when a **service** or **dependency** becomes unavailable or starts failing.

**Analogy:** Electrical Circuit Breaker



## ★ How it works (3 States)

### ① CLOSED (Normal)

- Requests flow normally.
- Failures are monitored.
- If failure rate exceeds configured threshold, breaker **opens**.



All Good!

### ② OPEN (Failing)

- Requests are immediately rejected or routed to **fallback**.
- Prevents wasting resources on calls likely to fail.
- After **timeout** period, moves to **HALF-OPEN**.



Not Good!

### ③ HALF-OPEN (Recovery Check)

- A **limited** number of requests are allowed through.
- If they succeed → **CLOSES**.
- If they fail → **OPENS** again.



Let's Test...

Failure rate  
exceeds  
threshold

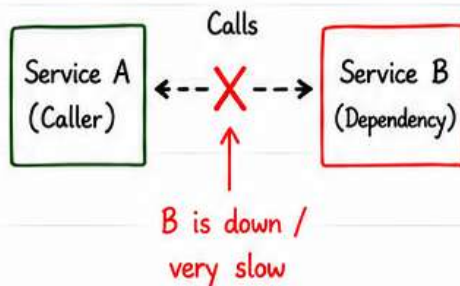
After  
timeout  
30  
sec

Success!

## ★ Example Scenario

### Without Circuit Breaker ☹️

- Service B becomes slow.
- Requests pile up.
- Threads, connections & resources are exhausted.
- Service A may also fail.



### With Circuit Breaker 😊

- After repeated failures, breaker **OPENS**.
- Service A stops calling B temporarily.
- Users get **fallback** / cached data / default response.
- System remains **responsive**!

Main logic →

Fallback method →

## ★ Example (Java + Resilience4j)

```

@CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
public PaymentResponse processPayment(PaymentRequest request) {
    return paymentClient.process(request);
}

public PaymentResponse fallback(PaymentRequest request, Exception ex) {
    return new PaymentResponse("Service unavailable");
}
  
```





# DATA CONSISTENCY IN MICROSERVICES



In Microservices, each service has its own database.

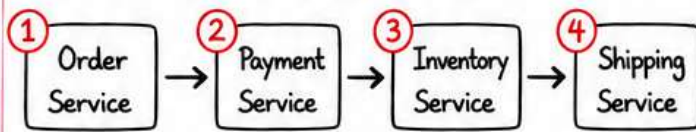
We **AVOID** distributed **2PC** transactions.

★ Instead we use patterns for **eventual consistency**.

## ① SAGA PATTERN (Most Common)

A Saga is a sequence of **local transactions**. Each service updates its DB and publishes an **event** for the next service.

Example: **E-commerce Order Flow**



If payment fails **X**

- Cancel the order
- Release reserved inventory

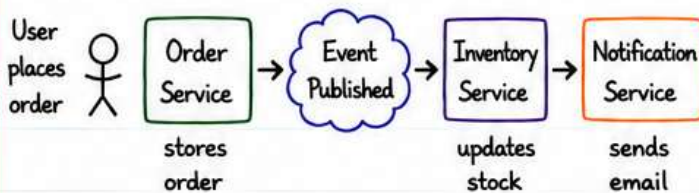


### Types of Saga

- **Choreography** → Services talk via events
- **Orchestration** → Central Saga Orchestrator controls the flow

## ② EVENTUAL CONSISTENCY

Instead of immediate consistency, services reach a consistent state **over time**.



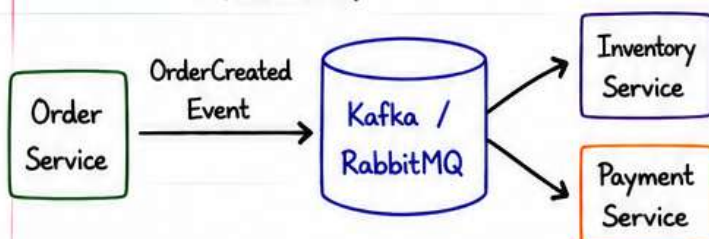
All services eventually reach the same state. ✓

## ③ EVENT-DRIVEN ARCHITECTURE

Services communicate through an **event broker**.

Examples:

- Apache Kafka
- RabbitMQ



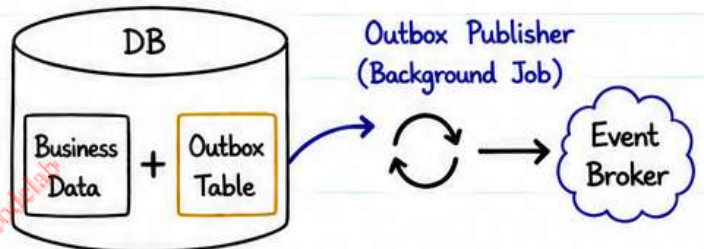
## ④ OUTBOX PATTERN

Problem:

- DB update succeeds
  - But event publishing fails
- 
- ```
graph LR; DB[(DB)] -- "X" --> EB((Event Broker));
```

Solution:

- Save the event in an **OUTBOX TABLE** within the same transaction.
- A separate process publishes events from the Outbox.



★ Ensures events are not lost!

## ⑤ IDEMPOTENCY

Messages can be delivered **multiple times**.

We must make operations **safe to repeat**.

Example:



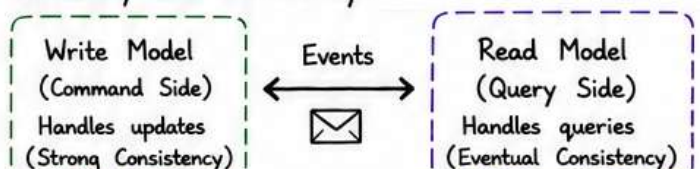
How?

- Use unique Transaction IDs
- Use Request IDs / **Idempotency Keys**
- Check before processing

No double charge!

## ⑥ CQRS (When Needed)

Separate read and write models for better scalability and consistency.

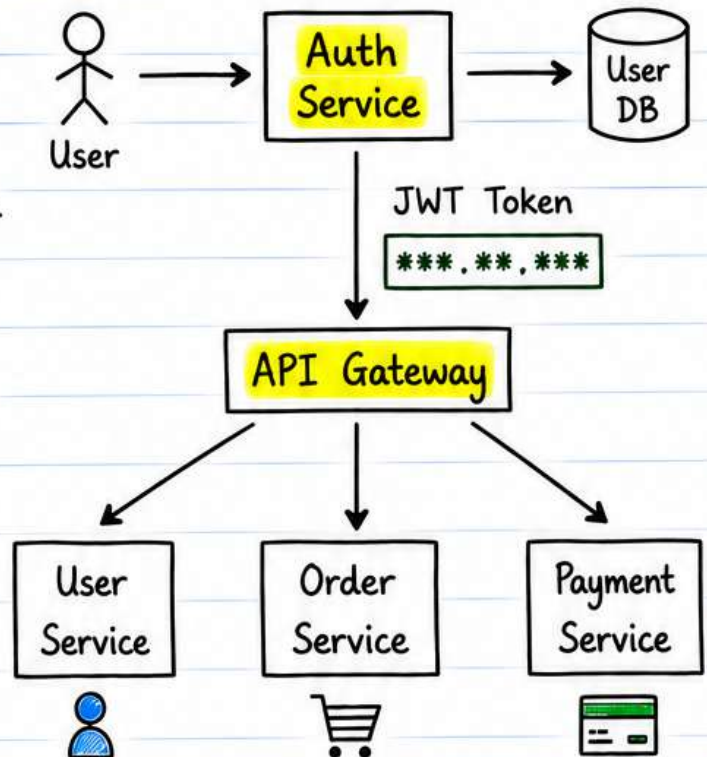


# AUTHENTICATION IN MICROSERVICES

★ **Authentication** = process of **verifying the identity** of a **user** or **service** before allowing access to resources.

## ① HOW DOES IT WORK?

- ① User logs in through the application.
- ② **Authentication Service** verifies credentials.
- ③ **Authentication Service** generates a **token (JWT)**.
- ④ User sends the **token** with every request.
- ⑤ **API Gateway** validates the token.
- ⑥ Request is forwarded to the appropriate microservice.
- ⑦ **Microservice** processes the request and returns a response.



### FLOW (Quick View)

User → Auth Service → JWT  
→ API Gateway → Microservices  
→ Response 😊

## ② COMMON AUTHENTICATION METHODS

### ① JWT (JSON WEB TOKEN)

★ **Most Common**

- User logs in once.
- Server generates a signed JWT token.
- Client sends token in Authorization header.

Authorization: Bearer  
eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp1bmI6I3E6Iik...

**Advantages:** ❤️

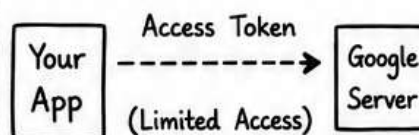
- ✓ Stateless
- ✓ Fast
- ✓ Scalable
- ✓ No session storage required

### ② OAUTH 2.0

Used when users log in using:

- Google 
- Facebook 
- GitHub 

Provides secure delegated access.



### ③ OPENID CONNECT (OIDC)

Built on top of **OAuth 2.0**.

Used to **verify user identity** and obtain user information.



## ③ WHY API GATEWAY?

- **API Gateway** validates the token once.
- Only **authenticated** requests reach services.
- Centralized security.
- Simplifies microservice code (less auth logic).



## ④ KEY TERMS (DON'T CONFUSE!)

- **Authentication** = Who are you?
- **Authorization** = What can you access?
- **JWT** = Token containing user information.
- **OAuth 2.0** = Third-party login authorization.
- **API Gateway** = Central entry point for security & routing.



# API GATEWAY vs LOAD BALANCER



★ **API GATEWAY**: Acts as a single entry point for all client requests.

It handles routing, security, rate limiting, logging, caching, monitoring, request/response transformation, etc.

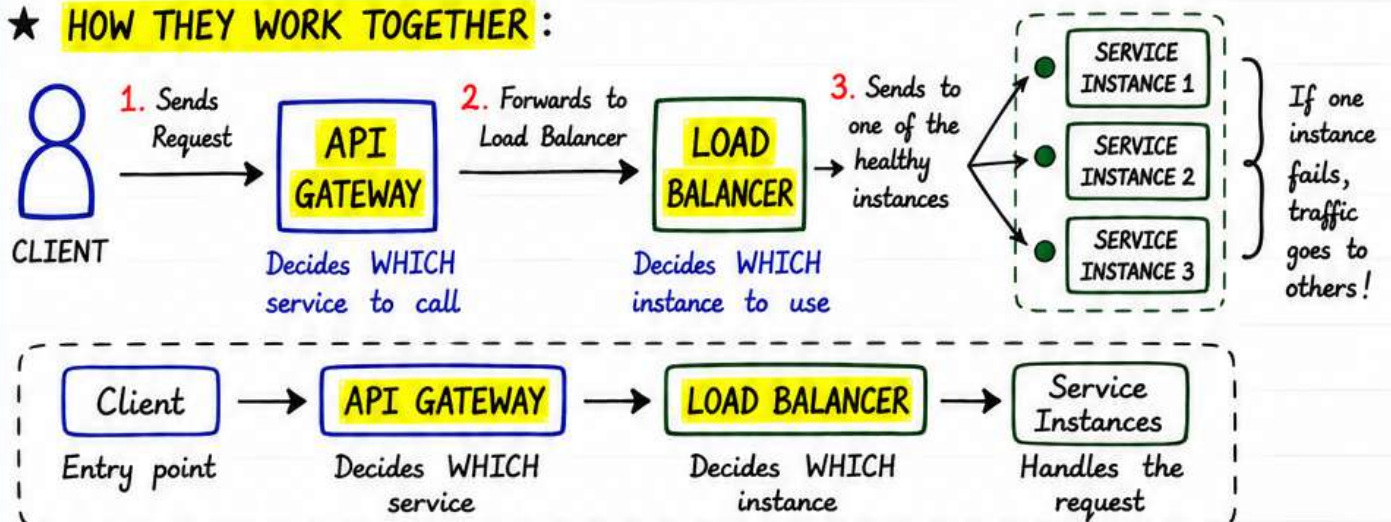


★ **LOAD BALANCER**: Distributes incoming traffic across multiple instances of a service to ensure availability, scalability and high performance.

## ★ KEY DIFFERENCES :

|   | FEATURE                         | API GATEWAY                                                    | LOAD BALANCER                                |
|---|---------------------------------|----------------------------------------------------------------|----------------------------------------------|
| ① | Purpose                         | Manages and routes API requests                                | Distributes traffic across multiple servers  |
| ② | Works At                        | Application / API level                                        | Layer 4 or Layer 7                           |
| ③ | Main Job                        | Single entry point for microservices                           | Prevent server overload                      |
| ④ | Routing                         | Routes based on paths, headers, query params, methods, etc.    | Routes to available healthy instances        |
| ⑤ | Security                        | Handles authentication, authorization, API keys, rate limiting | Usually does not handle authentication       |
| ⑥ | Request/Response Transformation | Can modify request / response                                  | Generally does not modify requests           |
| ⑦ | Caching                         | Supports response caching                                      | Usually no caching                           |
| ⑧ | Example Tools                   | Kong, Spring Cloud Gateway, AWS API Gateway, Apigee            | Nginx, HAProxy, AWS ELB, Azure Load Balancer |

## ★ HOW THEY WORK TOGETHER :





redis



# REDIS CACHING



Keep this page for quick revision! ★

→ **Redis** = Remote Dictionary Server

It is an **in-memory** data store used as a **cache** to **improve application performance** by **reducing database calls**.

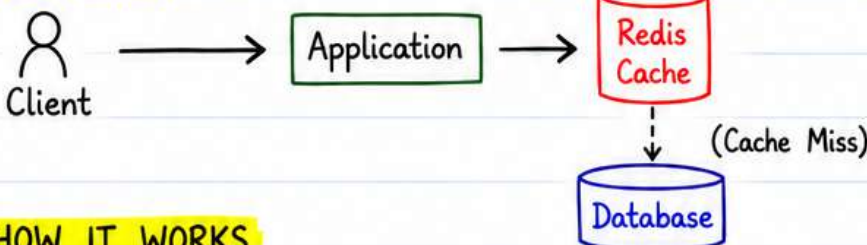
## 1 WHY REDIS CACHING?

Without cache:



Every request hits the database, increasing response time.

With Redis:

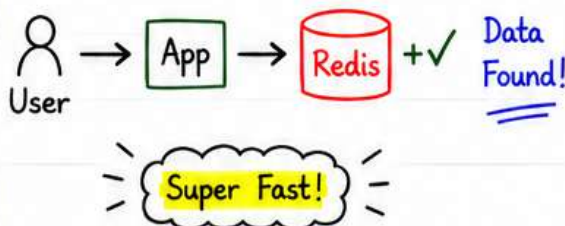


Most requests are served from Redis (**Fast**!)

## 2 HOW IT WORKS

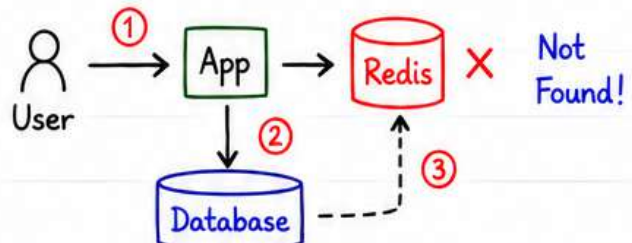
### A) CACHE HIT

- ① User requests data.
- ② Application checks Redis.
- ③ Data found in Redis.
- ④ Response returned **immediately**. 😊



### B) CACHE MISS

- ① Data not found in Redis.
- ② Application fetches data from DB.
- ③ Stores data in Redis.
- ④ Returns response to user.



## 3 BENEFITS

- **Faster** response time
- **Reduced** database load
- **Better** scalability
- **High** throughput
- Supports **distributed** caching



## 4 COMMON REDIS DATA TYPES

- String
- Hash
- Sorted Set
- List
- Set

Redis is very flexible! 😊

## 5 SPRING BOOT EXAMPLE

```

@Cacheable("students")
public Student getStudent(Long id) {
    return studentRepository.findById(id)
        .orElse(null);
}
  
```

- First call
- **Database** hit
- Next calls
- **Redis** cache hit

## INTERVIEW Q&A

Q: Why Redis is faster than a database?

A: Redis stores data in **RAM** (memory), while databases usually read data from **disk**. Memory access is much faster than disk access.





# IDEMPOTENCY IN MICROSERVICES



★ **Idempotency** → Means performing the **same operation** multiple times produces the **same result** as performing it **once**.

In microservices, idempotency is important because **network failures**, **retries**, and **duplicate requests** can occur frequently.



## 1 EXAMPLE WITHOUT IDEMPOTENCY

```
POST /payments
{
  "amount": 1000
}
```

Client sends request but doesn't get response (network issue)

↓  
Client retries the request

### Result:

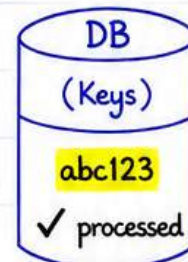
- First request → ₹1000 deducted
- Second request → ₹1000 deducted again

Total deduction = ₹2000 ❌

## 2 EXAMPLE WITH IDEMPOTENCY

```
POST /payments
Idempotency-Key: abc123
{
  "amount": 1000
}
```

- 1 Service receives request
- 2 Stores key **abc123** and processes payment
- 3 Response returned



If the same request comes again with key **abc123**:

- 4 Service checks the key
- 5 Finds it already processed
- 6 Returns previous response instead of processing again

Result:  
Payment deducted only once ✓  
😊

## 3 WHY IS IT IMPORTANT?



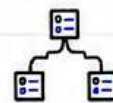
### 1 Network Failures

Requests may be retried automatically.



### 2 Message Queues

Systems like Kafka, RabbitMQ can deliver duplicate messages.



### 3 Distributed Systems

Multiple services may process the same request accidentally.



### 4 Payment Systems

Avoid duplicate transactions.

## 4 COMMON IMPLEMENTATION APPROACHES

### 1 IDEMPOTENCY KEY

Client sends a unique key.  
Store key in DB.

| Key    | Status    |
|--------|-----------|
| abc123 | Processed |

If key exists → return old response.



### 2 UNIQUE DB CONSTRAINT

Use unique constraint to prevent duplicates.

```
CREATE UNIQUE INDEX
idx_order_id
ON orders (order_id);
```

Duplicate order insertion will fail.



### 3 MESSAGE DEDUPLICATION

Store processed message IDs.

```
if (processedMessageIds
    .contains(messageId)) {
    return;
}
```

If already processed → ignore it.

